

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Программная инженерия»

СОГЛАСОВАНО

Департамент программной
инженерии ФКН НИУ ВШЭ
Научный руководитель,
кандидат технических наук,
доцент департамента
программной инженерии
факультета компьютерных наук

_____ / Н.С. Белова /
«__» _____ 2026 г.

УТВЕРЖДАЮ

Академический руководитель
образовательной программы
«Программная инженерия»,
старший преподаватель
департамента программной
инженерии

_____ / Н.А. Павлочев /
«__» _____ 2026 г.

**ОЧЕРЕДЬ СООБЩЕНИЙ НА ЯЗЫКЕ GO С ГАРАНТИЯМИ ДОСТАВКИ И
ГОРИЗОНТАЛЬНЫМ МАСШТАБИРОВАНИЕМ**

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.02-06 81 01–1 ЛУ

Исполнители:
студент группы БПИ233

_____ / Б.А. Багавиев /
«__» _____ 2026 г.

Москва 2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подл.	

УТВЕРЖДЕН
RU.17701729.02-06 81 01–1 ЛУ

**ОЧЕРЕДЬ СООБЩЕНИЙ НА ЯЗЫКЕ GO С ГАРАНТИЯМИ ДОСТАВКИ И
ГОРИЗОНТАЛЬНЫМ МАСШТАБИРОВАНИЕМ**

Пояснительная записка

RU.17701729.02-06 81 01–1

Листов 21

Инов.№ подл.	Подп. и дата	Взам. инв.№	Инов.№ дубл.	Подп. и дата

АННОТАЦИЯ

Настоящий документ «Пояснительная записка» разработан в соответствии с ГОСТ 19.404–79 и содержит обоснование принятых технических решений при разработке программного изделия «Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием» (BunnyMQ).

Документ описывает назначение и область применения разработанных компонентов, алгоритмы функционирования сегментированного журнала (`internal/storage`), конечных автоматов Raft (`internal/partition`, `internal/metadata`), координаторов кластера и групп потребителей, протокол Data API, а также обоснование выбора технических и программных средств (Go 1.25, dragonboat v4, gRPC, Protobuf, Prometheus, go.uber.org/zap) и ожидаемые показатели производительности.

Документ составлен в соответствии с требованиями ГОСТ 19.105–78 и ГОСТ 19.404–79.

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ	1
2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ	2
2.1. Назначение программы	2
2.2. Область применения	2
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ	3
3.1. Постановка задачи на разработку	3
3.1.1. Описание задачи	3
3.1.2. Описание применяемых методов	3
3.1.3. Ограничения и допущения	5
3.2. Описание алгоритма и функционирования программы	5
3.2.1. Архитектура узла	5
3.2.2. Подсистема хранения (internal/storage)	6
3.2.3. FSM метаданных (MetadataFSM)	7
3.2.4. FSM партии (PartitionFSM)	8
3.2.5. ClusterCoordinator (internal/coordinator/cluster)	8
3.2.6. DataCoordinator (internal/coordinator/data)	9
3.2.7. GroupCoordinator (internal/coordinator/group)	9
3.2.8. gRPC API серверы	10
3.3. Описание и обоснование выбора метода организации входных и выходных данных	10
3.3.1. Входные данные	10
3.3.2. Выходные данные	11
3.4. Описание и обоснование выбора состава технических и программных средств	11
3.4.1. Язык программирования: Go 1.25	11
3.4.2. Raft-библиотека: dragonboat v4	11
3.4.3. Транспорт: gRPC + Protocol Buffers	12
3.4.4. Мониторинг: Prometheus	12
3.4.5. Логирование: go.uber.org/zap	12
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ	14
4.1. Показатели производительности	14
4.2. Показатели надёжности	14
4.3. Показатели ресурсоёмкости	15
4.4. Экономические показатели	15
5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ	16

1. ВВЕДЕНИЕ

Наименование программы: **«Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием»** (сокращённое обозначение — BunnyMQ).

Условное обозначение темы разработки по ГОСТ 19.103-77: **RU.17701729.02-06.**

Разработка выполняется на основании технического задания «Очередь сообщений на языке Go с гарантиями доставки и горизонтальным масштабированием. Техническое задание» (обозначение: RU.17701729.02-06 ТЗ 01–1), принятого в 2026 г. на факультете компьютерных наук Национального исследовательского университета «Высшая школа экономики».

Исполнитель разработки — студент образовательной программы «Программная инженерия» факультета компьютерных наук НИУ ВШЭ, группа БПИ233, Б.А. Багавиев.

В состав программного изделия входят следующие компоненты:

1. `bunnymq` — серверный процесс брокера; запускается на каждом узле кластера;
2. `bunnymq-cli` — утилита командной строки для административных операций (создание тем, публикация, чтение, управление смещениями);
3. `pkg/client` — публичная Go-библиотека для встраивания в код производителей и потребителей.

Настоящая пояснительная записка охватывает всю серверную часть BunnyMQ: подсистему хранения сегментированного журнала, FSM Raft, координаторы, серверы gRPC API, аутентификационный слой и подсистему метрик.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

2.1. Назначение программы

BunnyMQ — распределённый брокер сообщений, обеспечивающий надёжную асинхронную передачу данных между компонентами распределённых приложений. Система реализует модель «темы → партиции → смещения», аналогичную Apache Kafka, с Raft-репликацией на уровне каждой партиции.

Компоненты системы выполняют следующие функции:

- `bunnymq` — принимает gRPC-запросы производителей и потребителей, применяет команды через Raft FSM, хранит данные сегментированного журнала на локальном диске каждого узла;
- `bunnymq-cli` — предоставляет административный интерфейс для управления кластером, темами и смещениями без необходимости писать клиентский код;
- `pkg/client` — инкапсулирует балансировку по лидеру, механизм долгого опроса и работу с группами потребителей, предоставляя прикладному коду высокоуровневый Go API.

2.2. Область применения

BunnyMQ ориентирован на применение в инфраструктуре backend-приложений, где требуется: декуплирование компонентов через очередь сообщений, сохранение порядка сообщений внутри партиции, воспроизведение событий за произвольный временной интервал, горизонтальное масштабирование пропускной способности за счёт партиционирования, гарантированная доставка при отказе узла.

Типичные сценарии: сбор телеметрии и логов приложений, системы обработки заказов в электронной коммерции, конвейеры потоковой обработки данных, децентрализованные очереди задач.

Целевые пользователи: backend-разработчики, встраивающие `pkg/client` в прикладной код; DevOps-инженеры, развёртывающие кластер и настраивающие `bunnymq-cli`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку

3.1.1. Описание задачи

Требуется разработать распределённый брокер сообщений, реализующий следующий функциональный минимум:

1. управление темами: создание, удаление, изменение числа партиций и политики хранения;
2. публикация пакетов сообщений с выбором гарантии доставки: `acks=0` (без ожидания) или `acks=all` (после подтверждения кворума);
3. чтение пакетов по смещению с долгим опросом (long-poll) при отсутствии новых данных;
4. получение смещений: наиболее раннее, наиболее позднее, первое за заданную временную метку;
5. управление группами потребителей: вступление, выход, heartbeat, фиксация смещений, чтение зафиксированных смещений;
6. описание кластера: перечень узлов с ролями (LEADER / FOLLOWER);
7. опциональная токен-аутентификация gRPC-запросов.

Дополнительные нефункциональные требования: корректное восстановление после сбоя (torn write recovery), периодическое удаление устаревших данных по политике хранения, метрики Prometheus, структурированное логирование.

3.1.2. Описание применяемых методов

Сегментированный append-only журнал. Данные каждой партиции хранятся в виде последовательности сегментов. Каждый сегмент — тройка файлов с одинаковым именем (значение базового смещения первого пакета, дополненное нулями до 20 знаков): `.log` для самих пакетов, `.index` для индекса смещений, `.timeindex` для временного индекса. При записи новых данных используется последовательная запись с флагами `O_APPEND|O_WRONLY`, что обеспечивает максимальную пропускную способность дискового ввода-вывода. Индексы хранятся в mmap-отображениях: каждая запись `OffsetIndexSegment` занимает ровно 8 байт (4 байта относительного

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

смещения и 4 байта позиции в `.log`-файле в формате `big-endian`), каждая запись `TimeIndexSegment` — 12 байт (8 байт временной метки и 4 байта относительного смещения). Поиск по смещению выполняется за $O(\log n)$ через двоичный поиск по индексу.

Формат пакета. Физический формат данных на диске совпадает с форматом по сети. Заголовок пакета занимает 38 байт и содержит следующие поля: `base_offset` (8 байт), `batch_length` (4 байта), `record_count` (4 байта), `crc32` (4 байта, CRC-32C по алгоритму Castagnoli с полиномом `0x1EDC6F41` над полем `records[]`), `attributes` (2 байта), `base_timestamp` (8 байт), `max_timestamp` (8 байт). Записи внутри пакета кодируются с переменной длиной через `varint`. Совпадение формата диска и сети исключает промежуточную сериализацию при передаче данных потребителю.

Raft-репликация через dragonboat. Для обеспечения согласованности применяется алгоритм Raft, реализованный библиотекой `dragonboat v4`. На каждом узле работает единственный `NodeHost`, который содержит один `chard` метаданных (`shard ID = 0`) и по одному `shard`'у для каждой реплики партиции на данном узле. Гарантия доставки `acks=0` реализована через `NodeHost.Propose` (асинхронно), `acks=all` — через `NodeHost.SyncPropose` (блокируется до фиксации записи кворумом).

Детерминизм FSM. Оба FSM (`MetadataFSM` и `PartitionFSM`) не вызывают `time.Now()` и `rand` напрямую. Все нефиксированные значения (временные метки, UUID участников группы) передаются в теле команды от координатора и применяются FSM'ом как есть. Это обязательное свойство для корректного воспроизведения Raft-журнала на репликах.

Долгий опрос (long-poll). Для снижения задержки доставки сообщений потребителю без постоянного поллинга применяется следующий механизм: после каждого успешного `Append` объект `Storage` заменяет канал `newDataCh` на новый (закрытый) экземпляр. Компонент `DataCoordinator`, обрабатывающий запрос `Fetch`, сначала берёт снимок `ch := storage.NewDataCh()`, затем выполняет `Read`. Если данных нет, координатор ждёт `select { case <-ch: ... case <-ctx.Done(): ... }`. При появлении новых данных канал закрывается, `select` разблокируется и `Read` повторяется. Максимальное время ожидания ограничено параметром `max_wait_ms` из запроса.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Алгоритм назначения партиций в группе потребителей (Range Assignment).

При ребалансировке группы координатор формирует для каждой темы список партиций, отсортированных по `partition_id`, и список участников, отсортированных по `member_id`. Партиции делятся на непрерывные диапазоны: базовый размер диапазона — $l \lfloor \frac{P}{M} r \rfloor$, где P — число партиций, M — число участников; первые $P \bmod M$ участников получают на одну партицию больше. Алгоритм идентичен реализации в Apache Kafka и гарантирует детерминированное распределение без дополнительного координирующего состояния.

Torn write recovery. При каждом успешном `Append FSM` партиции записывает контрольную точку в файл `applied.idx`, содержащий последнее применённое смещение Raft-записи. При следующем запуске (`IOndiskStateMachine.Open`) сравниваются базовое смещение последнего пакета в журнале (`Storage.LatestOffset`) с зафиксированным значением в `applied.idx`. Если в журнале обнаружены данные сверх последней подтверждённой контрольной точки, они удаляются методом `Storage.TruncateTo`.

3.1.3. Ограничения и допущения

- Гарантия `acks=1` не реализована намеренно: частичная запись без кворума создаёт риски потери данных без существенного выигрыша по задержке по сравнению с `acks=0`.
- В версии `v1` снимки Raft (`dragonboat snapshots`) отключены (`SnapshotEntries = 1 << 62`); восстановление FSM выполняется воспроизведением полного Raft-журнала.
- Добавление узлов в работающий кластер не поддерживается: список членов фиксирован в конфигурационном файле.
- Транзакционная запись (`idempotent producer`) не реализована.

3.2. Описание алгоритма и функционирования программы

3.2.1. Архитектура узла

Каждый узел брокера объединяет следующие компоненты:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

```

cmd/bunnymq/
└─ main.go          - инициализация, graceful shutdown

internal/
├─ config/          - загрузка YAML-конфигурации
├─ raft/            - обёртка NodeHost (dragonboat)
├─ metadata/        - MetadataFSM (IStateMachine, shard 0)
├─ partition/       - PartitionFSM (IOOnDiskStateMachine, shard 1...
N)
├─ storage/         - сегментированный журнал
├─ coordinator/
│   ├── cluster/    - ClusterCoordinator
│   ├── data/       - DataCoordinator
│   └─ group/       - GroupCoordinator
├─ api/
│   ├── data/       - gRPC-сервер DataService
│   └─ management/ - gRPC-сервер ManagementService
├─ auth/            - interceptor токен-аутентификации
├─ metrics/         - регистрация метрик Prometheus
└─ log/             - инициализация zap-логгера

```

3.2.2. Подсистема хранения (internal/storage)

Объект `Storage` управляет упорядоченным срезом `[]SegmentStorage`. Последний элемент — активный (writable), остальные — запечатаны (read-only). При вызове `Append(batch []byte) Storage`:

1. записывает актуальное `base_offset` в первые 8 байт переданного пакета (без инвалидации CRC, так как контрольная сумма покрывает только записи начиная с байта 38);
2. передаёт пакет активному `SegmentStorage.Append`;
3. если размер `.log`-файла достиг `segment_max_bytes` — запечатывает сегмент и создаёт новый с `base_offset = nextOffset`;
4. закрывает текущий `newDataCh` и заменяет его новым.

Чтение (`Read(offset int64, maxBytes int)`) выполняется следующим образом:

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

1. двоичный поиск по срезу сегментов находит тот, чей `base_offset ≤ offset`;
2. внутри сегмента двоичный поиск по `.index`-файлу (`mmap`) находит ближайшую запись и определяет позицию начала чтения в `.log`-файле;
3. линейное чтение пакетов до исчерпания `maxBytes` или достижения конца сегмента.

Запечатывание сегмента: индексные файлы усекаются до фактического размера, вызывается `msync`, файлы `remapped` в `PROT_READ`, `.log`-файл переоткрывается в `O_RDONLY`, вызывается `fsync`.

Фоновая горутинa удаления выполняется каждую минуту. Сегмент удаляется, если выполняется хотя бы одно из условий: `max_timestamp < now - retention_ms` либо суммарный объём всех сегментов превышает `retention_bytes`. Активный сегмент не удаляется никогда.

3.2.3. FSM метаданных (MetadataFSM)

`MetadataFSM` реализует интерфейс `IStateMachine dragonboat` и хранит полное состояние кластера в памяти: топологию тем и партиций, состояние групп потребителей, регистрацию узлов. Состояние сериализуется в JSON при создании снимка.

Ключевые атрибуты состояния:

- `Topics` — словарь `string → TopicMeta` (имя темы → метаданные: `partition_count`, `replication_factor`, `retention_ms`, `retention_bytes`, `created_at_ms`);
- `Partitions` — словарь `(topic, partition_id) → PartitionMeta` (`shard_id`, список реплик, `id` лидера, `leader_epoch`);
- `Nodes` — словарь `node_id → NodeInfo`;
- `Groups` — словарь `group_id → ConsumerGroupMeta` (поколение, участники, зафиксированные смещения);
- `NextShardID` — монотонно возрастающий счётчик; при создании темы из него выделяется `N` `shard`-идентификаторов (по числу партиций).

Команды FSM передаются в JSON-конверте `MetadataCommand` с кратким именем поля (например, "ct" для `CreateTopicCmd`). При команде `CreateTopic`

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

FSM проверяет отсутствие темы, создаёт `TopicMeta`, для каждой партии создаёт `PartitionMeta` с назначенным `ShardID` и увеличивает `NextShardID` на `N`. Все временные метки берутся из поля `created_at_ms` команды.

Запросы к FSM (`IStateMachine.Lookup`) обрабатываются напрямую без прохождения через Raft, что обеспечивает чтение без накладных расходов на репликацию. Запросы, требующие строгой линейаризуемости (описание кластера), выполняются через `NodeHost.ReadLocalNode` с проверкой актуальности лидерства.

3.2.4. FSM партии (PartitionFSM)

`PartitionFSM` реализует интерфейс `IOndiskStateMachine dragonboat`. Один экземпляр управляет одной репликой одной партии.

Открытие (Open): создаётся объект `Storage` для каталога `<data_dir>/partitions/<topic>-<partition_id>`; читается файл `applied.idx`; если `Storage.LatestOffset()` превышает сохранённое значение, вызывается `Storage.TruncateTo` для удаления незаписанных в контрольную точку пакетов. Метод возвращает последний применённый индекс Raft-записи.

Применение команды (Update): единственная команда — `AppendBatch`. FSM вызывает `Storage.Append(batch)`, затем `Storage.Sync()`, затем обновляет `applied.idx`. Порядок принципиален: если процесс упадёт после `Append` и до записи `applied.idx`, следующий запуск обнаружит расхождение и усечёт журнал.

Запрос без репликации (Lookup): возвращает результат `Storage.Read`, `Storage.EarliestOffset` или `Storage.LatestOffset` без обращения к Raft.

3.2.5. ClusterCoordinator (internal/coordinator/cluster)

`ClusterCoordinator` отвечает за жизненный цикл кластера: старт `shards dragonboat`, регистрацию узла через `RegisterNode`-команду к `MetadataFSM`, отслеживание смен лидера партий. Лидерство определяется вызовом `dragonboat.NodeHost.GetLeaderID(shardID)` без дополнительных сетевых запросов.

При обнаружении смены лидера (через `LeaderObserver dragonboat`) координатор предлагает `AssignPartitionLeader`-команду к `MetadataFSM`, обновляя поле `leader_node_id` и инкрементируя `leader_epoch`.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.2.6. DataCoordinator (internal/coordinator/data)

DataCoordinator маршрутизирует запросы Data API к нужному узлу:

- если текущий узел является лидером партиции, команда передаётся напрямую в PartitionFSM через raftHost;
- иначе возвращается ответ NotLeader с адресом лидера из MetadataFSM; клиент автоматически переключается.

Алгоритм обработки Fetch:

1. DataCoordinator получает смещение offset и параметр max_wait_ms из запроса;
2. берёт снимок `ch := storage.NewDataCh()`;
3. выполняет `storage.Read(offset, maxBytes)` через `PartitionFSM.Lookup`;
4. если данных нет, ожидает `select { case <-ch: <повтор Read> case <-timer.C: <пустой ответ> case <-ctx.Done(): }`.

3.2.7. GroupCoordinator (internal/coordinator/group)

GroupCoordinator реализует протокол управления группами потребителей через MetadataFSM.

JoinGroup: координатор генерирует `member_id` (UUID) и `session_timeout_ms` на стороне сервера, передаёт их в команде `JoinConsumerGroupCmd`. FSM добавляет участника в состояние группы. Если группа не существует, FSM создаёт её с `generation_id = 0`.

Heartbeat: команда `HeartbeatConsumerGroupCmd` обновляет поле `last_heartbeat_ms` в `MemberInfo`. Координатор запускает фоновую горутину, которая каждые 5 секунд проверяет все группы: участники с `last_heartbeat_ms + session_timeout_ms < now` удаляются командой `LeaveConsumerGroupCmd`, после чего инициируется ребалансировка.

Ребалансировка: вызов `RebalanceConsumerGroupCmd` переводит группу в новое поколение (`generation_id++`), пересчитывает назначения партиций по алгоритму Range Assignment и записывает результат в

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

`MemberInfo.AssignedPartitions`. Все последующие вызовы `JoinGroup` от членов группы возвращают актуальное назначение из нового поколения.

3.2.8. gRPC API серверы

`ManagementService` и `DataService` реализованы как gRPC-серверы на двух портах (по умолчанию 9091 и 9090 соответственно). Для каждого метода сервера существует `interceptor` аутентификации: при наличии непустого списка `auth.tokens` в конфигурации `interceptor` извлекает заголовок `bunmq-auth-token` из `metadata` gRPC-запроса и сравнивает с допустимыми токенами. При несовпадении возвращается статус `UNAUTHENTICATED`.

Метрики HTTP-сервера Prometheus доступны на третьем порту (по умолчанию 9092) по пути `/metrics`.

3.3. Описание и обоснование выбора метода организации входных и выходных данных

3.3.1. Входные данные

gRPC-запросы. Все клиентские взаимодействия выполняются через gRPC поверх HTTP/2 с кодированием Protobuf. Входные данные строго типизированы схемами `.proto`-файлов (пакет `bunmq.v1`). Пакеты сообщений передаются в поле `records bytes` в сыром байтовом формате, совпадающем с форматом хранения на диске (38-байтный заголовок + `varint`-кодированные записи).

Конфигурационный файл. Конфигурация узла задаётся файлом YAML, путь к которому передаётся флагом `--config`. Обязательные параметры: `node_id`, `raft_address`, `data_dir`. Конфигурация загружается однократно при старте и не перечитывается в рантайме.

Сигналы операционной системы. `SIGTERM` и `SIGINT` иницируют `graceful shutdown`: прекращается приём новых RPC, дожидается завершения текущих запросов, Storage синхронизируется на диск, `NodeHost dragonboat` закрывается.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

3.3.2. Выходные данные

gRPC-ответы. Ответы возвращаются в формате Protobuf. Пакеты данных при Fetch передаются в поле `records bytes` без промежуточной сериализации — та же байтовая последовательность, что хранится на диске.

Метрики Prometheus. Счётчики и гистограммы в формате `text/plain` (OpenMetrics) по HTTP.

Структурированные логи. JSON-строки на `stdout`, уровень и поля определяются параметрами `log.level` конфигурации.

3.4. Описание и обоснование выбора состава технических и программных средств

3.4.1. Язык программирования: Go 1.25

Выбор языка Go обоснован следующим:

- нативная горутинная модель конкурентности подходит для одновременного обслуживания сотен gRPC-соединений без создания отдельного потока на каждое;
- сборщик мусора с низкими паузами не вносит неконтролируемых задержек в критический путь записи;
- стандартная библиотека предоставляет готовые пакеты для работы с файлами, `mmap` (`syscall.Mmap`), сигналами и контекстами отмены;
- статическая компиляция без внешних зависимостей упрощает развёртывание.

3.4.2. Raft-библиотека: dragonboat v4

Dragonboat выбрана среди Go-реализаций Raft по следующим причинам:

- поддержка Multi-Raft (несколько независимых групп Raft в одном процессе) необходима для изоляции партиций: сбой одного shard не блокирует другие;
- интерфейс `IOndiskStateMachine` позволяет FSM партиции использовать внешнее хранилище (сегментированный журнал) без обязательного snapshot-механизма dragonboat;

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

- `NodeHost.ReadLocalNode` позволяет читать FSM-состояние без прохождения через Raft-журнал (follower read не поддерживается в v1, но прямое чтение FSM лидера без репликации доступно);
- активное сообщество и стабильный API.

Альтернативы (etcd/raft, hashicorp/raft) требуют значительно большего объёма интеграционного кода и не поддерживают Multi-Raft из коробки.

3.4.3. Транспорт: gRPC + Protocol Buffers

gRPC выбран как транспортный уровень вместо REST по следующим причинам:

- двоичный формат Protobuf значительно компактнее JSON при передаче пакетов сообщений;
- HTTP/2 мультиплексирование снижает накладные расходы при параллельных запросах;
- строгая типизация `.proto`-схем исключает несогласованность между клиентом и сервером;
- встроенная потоковая передача (server-side streaming) пригодна для расширения API в будущем.

3.4.4. Мониторинг: Prometheus

Prometheus выбран как система сбора метрик:

- pull-модель не требует изменений в сервисе при добавлении новых scrapers;
- клиентская библиотека `prometheus/client_golang` предоставляет готовые типы метрик (Counter, Gauge, Histogram) с атомарными операциями;
- интеграция с `dragonboat`: `EnableMetrics: true` в конфигурации `NodeHostConfig` регистрирует внутренние метрики `dragonboat` в глобальном реестре Prometheus.

3.4.5. Логирование: go.uber.org/zap

Структурированное логирование через zap выбрано из-за минимальных накладных расходов на форматирование: zap избегает рефлексии при сборке JSON-строк, что

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

критично на горячем пути записи. Уровень логирования (`debug` | `info` | `warn` | `error`) настраивается через конфигурационный файл.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Показатели производительности

Ожидаемые показатели производительности получены на основании бенчмарков дисковой подсистемы (`benchmarks/disk_test.go`) и оценок на основании характеристик `dragonboat v4`.

Таблица 4.1 — Ожидаемые показатели производительности BunnyMQ

Показатель	Значение	Примечание
Пропускная способность записи (<code>acks=all</code> , 1 партиция)	≥ 50 МБ/с	Ограничена скоростью диска и задержкой Raft (200 мс RTT)
Пропускная способность записи (<code>acks=0</code> , 1 партиция)	≥ 200 МБ/с	Ограничена только дисковым вводом-выводом
Задержка Produce (<code>acks=all</code> , p99, LAN)	≤ 50 мс	Два RTT Raft + <code>fsync</code>
Задержка Produce (<code>acks=0</code> , p99)	≤ 5 мс	Без ожидания кворума
Задержка Fetch (данные присутствуют, p99)	≤ 10 мс	Чтение с диска без репликации
Задержка уведомления потребителя (<code>long-poll</code>)	≤ 5 мс	Через <code>newDataCh</code> после <code>Append</code>
Минимальное число узлов для отказоустойчивости	3 узла	Raft-кворум: 2 из 3
Максимальный размер пакета	1 МиБ	Ограничение gRPC по умолчанию

4.2. Показатели надёжности

- Отказ одного узла кластера из трёх не нарушает доступность записи (кворум сохраняется).
- Torn write (незавершённая запись пакета) исправляется автоматически при перезапуске узла за счёт сравнения `applied.idx` с фактическим состоянием журнала.
- Минимальная единица восстановления после отказа — ракет целиком; частично записанные пакеты отбрасываются.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

4.3. Показатели ресурсоёмкости

Таблица 4.2 — Минимальные требования к аппаратным ресурсам узла

Ресурс	Требование
CPU	2 ядра x86-64; 4 ядра рекомендовано при нагрузке более 10 000 RPS
RAM	512 МБ без учёта данных mmap-индексов; mmap'ированные индексы вытесняются ОС при нехватке памяти
Диск	SSD рекомендован; объём — $\text{retention_bytes} \times \text{число партиций на узле} + 10\%$ на журналы Raft
Сеть	TCP-связность между всеми узлами; задержка < 100 мс для корректной работы Raft (ElectionRTT = 2 с при RTT = 200 мс)

4.4. Экономические показатели

Разработка выполнена в рамках учебного курсового проекта. Все используемые библиотеки и компоненты распространяются по открытым лицензиям (Go — BSD; dragonboat — Apache 2.0; gRPC-Go — Apache 2.0; Prometheus client_golang — Apache 2.0; zap — MIT), что исключает лицензионные расходы. Требуемая инфраструктура — от трёх виртуальных машин стандартной конфигурации (2 CPU, 1 ГБ RAM, 20 ГБ SSD), стоимость которых в публичных облаках составляет порядка 1 500–3 000 руб./мес. за узел.

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

1. ГОСТ 19.404–79. Единая система программной документации. Пояснительная записка. Требования к содержанию и оформлению. — М.: Стандартинформ, 2010.
2. ГОСТ 19.105–78. Единая система программной документации. Общие требования к программным документам. — М.: Стандартинформ, 2010.
3. ГОСТ 19.106–78. Единая система программной документации. Требования к программным документам, выполненным печатным способом. — М.: Стандартинформ, 2010.
4. ГОСТ 19.201–78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. — М.: Стандартинформ, 2010.
5. Ongaro D., Ousterhout J. In Search of an Understandable Consensus Algorithm (Extended Version). USENIX ATC „14, 2014. [Электронный ресурс]. — URL: <https://raft.github.io/raft.pdf> (дата обращения: 26.04.2026).
6. The Go Programming Language Specification. Версия 1.25. [Электронный ресурс]. — URL: <https://go.dev/ref/spec> (дата обращения: 26.04.2026).
7. dragonboat: A Multi-Group Raft library in Go. Документация dragonboat v4. [Электронный ресурс]. — URL: <https://github.com/lni/dragonboat> (дата обращения: 26.04.2026).
8. Apache Kafka Documentation. Версия 3.7. [Электронный ресурс]. — URL: <https://kafka.apache.org/documentation/> (дата обращения: 26.04.2026).
9. Kreps J., Narkhede N., Rao J. Kafka: a Distributed Messaging System for Log Processing. NetDB „11, 2011. [Электронный ресурс]. — URL: <https://notes.stephenholiday.com/Kafka.pdf> (дата обращения: 26.04.2026).
10. gRPC Documentation. [Электронный ресурс]. — URL: <https://grpc.io/docs/> (дата обращения: 26.04.2026).
11. Protocol Buffers Documentation. Версия proto3. [Электронный ресурс]. — URL: <https://protobuf.dev/> (дата обращения: 26.04.2026).

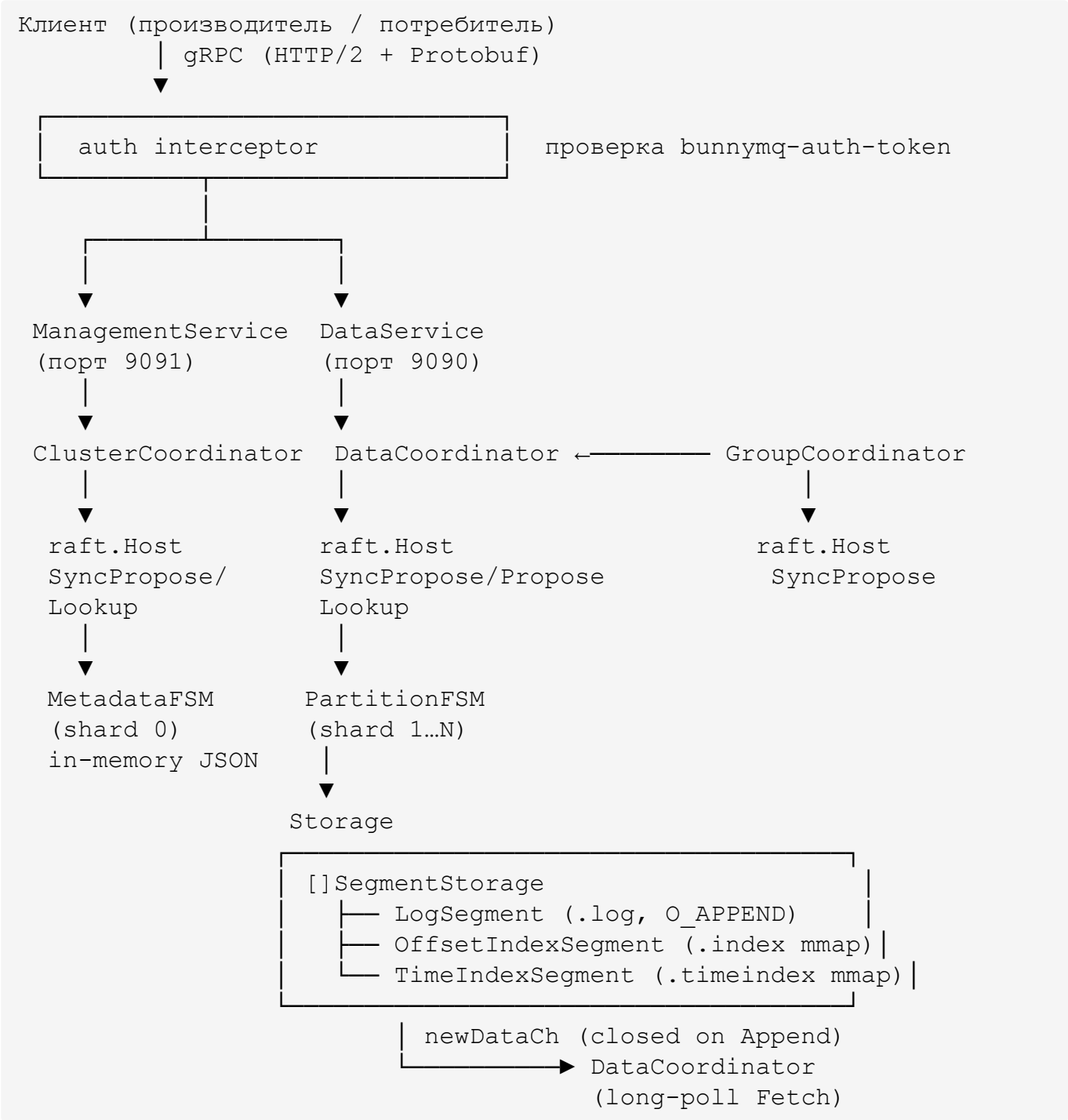
Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

12. Prometheus Documentation. [Электронный ресурс]. — URL: <https://prometheus.io/docs/> (дата обращения: 26.04.2026).
13. Uber go/zip. Blazing fast, structured, leveled logging in Go. [Электронный ресурс]. — URL: <https://github.com/uber-go/zip> (дата обращения: 26.04.2026).

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

СХЕМА ВЗАИМОДЕЙСТВИЯ КОМПОНЕНТОВ УЗЛА

Ниже приведена текстовая схема потоков данных внутри одного узла брокера BunnyMQ.



Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ФОРМАТ ПАКЕТА ДАННЫХ

Таблица П2.2 — Структура записи индекса смещений (.index, 8 байт/запись)

Байты	Тип	Поле
0–3	int32	<code>relative_offset</code> = <code>batch.base_offset</code> – <code>segment.base_offset</code>
4–7	int32	<code>position</code> — смещение начала пакета в .log-файле

Таблица П2.3 — Структура записи временного индекса (.timeindex, 12 байт/запись)

Байты	Тип	Поле
0–7	int64	<code>timestamp_ms</code> = <code>batch.max_timestamp</code>
8–11	int32	<code>relative_offset</code> = <code>batch.base_offset</code> – <code>segment.base_offset</code>

Изм.	Лист	№ докум.	Подп.	Дата
RU.17701729.02-06 81				
Инв. № подл.	Подп. и дата	Взам. инв. №	Инв. № дубл.	Подп. и дата

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

[illegible]